

접어서 넣어라, 흘러 읽지 말고: 소비자 기기 LLM 추론을 위한 상주(常駐) 최적화로서의 순환 깊이

Fold, Don't Stream: Recurrent Depth as a Storage-Residency Optimization for Consumer-Device LLM Inference

안기점 (Gigyeom Ahn)

독립 연구 — AI 연구 조수와의 협업으로 수행†

2026년 7월 27일 — 프리프린트 v3, 접힌 모델의 Metal 실행과 학습 없는 접기 품질의 실측 전장 추가; 4개 렌즈 적대적 리뷰 반영 개정 (영문판의 한국어 해석본)

초록

언어 모델이 소비자 기기의 RAM을 초과하면 디코딩은 저장 절벽(storage cliff)에서 떨어진다: 토큰마다 활성 파라미터를 디스크에서 다시 읽어야 한다. 우리는 이를 Apple M1 Max(32GB)에서 직접 측정한다 — 4비트 dense 70B는 생성 토큰당 SSD에서 40~44GB를 읽고 약 0.02 tokens/s로 디코딩한다. 순환 깊이(작은 층 스택을 R회 실행)는 이미 확립된 메커니즘이다; 우리는 이를 파라미터·추론 효율의 수단이 아니라 그 절벽에 맞서는 지렛대로 다루고, 그것이 사주는 것과 사주지 못하는 것을 함께 측정한다.

15M·42M 파라미터 통제 실험에서, 동일 상주 가중치 및 동일 학습 토큰 조건의 루프형 트랜스포머는 메모리를 2배로 늘려 얻는 품질 격차의 41%(dense)와 최대 54%(MoE)를 회복하며, 두 하드웨어·정밀도 스택에서 재현된다. 루프형 MoE 내부에서 sticky 라우팅 — 첫 패스의 expert 선택을 재사용 — 은 expert 트래픽을 top-k에 동결한 채 자유 재라우팅과 시드 잡음 안에서 일치하므로, 접근 지역성은 공짜다. 이어서 llama.cpp에 대한 그래프 접기 패치가 같은 노트북에서 절벽을 제거한다: dense 70B 약 120배, 118B-A8B MoE 약 1.7배, 스왑 0. 로더를 접힌 작업 집합에 맞추면 그 118B 체크포인트가 GPU에 올라가, 가중치 57.6GB가 wired 25.2GB로 34.4 tokens/s에 디코딩한다 — 다만 출력은 열화되어 있다.

이어서 그 열화를 한정한다. 표적 복구는 접힌 118B를 퍼플렉시티 4.2×10^4 에서 27.6까지 내리지만(접지 않으면 6.40) 그 이상은 아니다: 물리층의 전체 256-전문가 집합에 대한 토큰별 최소제공 — 어떤 라우팅·라우터 재적합·게이팅 어댑터에도 적용되는 하한 — 은 목표 잔차의 0.93~0.95를 남기며, 3072차원 속 임의의 256방향이 달성하는 우연 바닥은 0.957이다. 접기가 만드는 궤적 위에서 물리층의 전문가 기저는 이웃 층의 라우팅 함수에 대해 임의의 부분 공간보다 근소하게 나올 뿐이므로, 게이팅 쪽 복구로는 이 격차가 닫히지 않고 품질은 루프 네이티브 학습에서 와야 한다. 통제된 종착점으로, 프로덕션 루프 모델의 펼친 쌍둥이 — 함수와 출력이 구성상 동일 — 가 약 150배 속도 격차의 원인을 상주 하나로 분리해낸다.

1 서론

이 연구는 실패 보고서에서 시작되었다. 32GB 통합 메모리의 M1 Max에서 Qwen3.5-122B-A10B(4비트, 78.6GB)를 서빙하려 시도하며 우리는 소프트웨어 도구 상자를 소진했다 — 프리페치 심도 조절, mlock 페이지 고정, 명시적 expert 캐시, 세 종류의 투기적 디코딩, expert 수 축소, 스레드 스왑 — 어느 것도 디코드 속도를 1.5 tokens/s 너머로 움직이지 못했다. 라우터 계측이 이유를 설명했다: 약 300 토큰스텝 동안 전체 12,288개 (층, expert) 슬롯의 75%가 접촉되었고, 따라서 "핫셋"은 풀 전체이며, 기성 라우터에서는 어떤 캐싱 정책도 도움이 될 수 없다(expert 재사용을 집중시키는 사후 라우터 미세조정[12]은 이 경계를 움직일 수 있으나 우리는 평가하지 않았다). 병목은 연산이 아니라 토큰당 바이트다: 자기회귀 스텝은 활성 파라미터 전부를 읽어야 하며, 모델이 RAM을 초과하는 순간 그 읽기는 수십~수백 GB/s의 메모리 대신 약 2GB/s의 저장장치를 때린다.

수학적 탈출구는 둘뿐이다: 덜 읽거나(회소화), 같은 바이트를 여러 번 읽거나(상각). 본 논문은 후자의 깊이-축 버전을 전개한다: 순환 깊이(루프) 트랜스포머는 L개의 유일 층을 저장하고 이를 R회 실행하므로, 상주 풋프린트는 L층분 이면서 유효 깊이는 L·R이 된다. 루프 트랜스포머에 관한

선행 연구는 그 이득을 파라미터 효율[2,3]이나 테스트타임 추론[4,5,6]으로 프레임했다; 우리가 찾을 수 있었던 유일한 상주 논거는 MobileLLM[2]의 20MB SRAM에 관한 단락이다. 이 주장의 DRAM 대 SSD 버전 — 유일 가중치가 RAM에 들어가도록 스택을 접으면 저장 절벽이 사라진다 — 은 설계·실측된 적이 없어 보인다.

기여. (1) 사전 등록된 kill-gate를 갖춘 15M 규모 통제 품질 실험: 동일 상주 가중치·동일 학습 토큰에서 루프는 2배 메모리 품질 격차의 41%(dense)/최대 54%(MoE)를 회복하며, M1 Max/fp32와 NVIDIA T4/fp16에서 재현된다 (§3) — 연산량 일치 대조가 이 주장을 동일-토큰 체제로 한정한다 (§3.2). (2) 기존에 측정되지 않은 MoE 설계 축인 루프 내 라우팅 정책과, sticky 라우팅이 추가 expert 트래픽 0으로 자유 재라우팅과 시드 잡음 안에서 대등하다는 멀티시드 발견 (§3.3). (3) 수확 포화와 실용 최적점 R=3~4를 확립하는 R 스왑 (§3.4). (4) 대역폭 공학이 아니라 상주가 구속 변수임을 보이는 시스템 증거: 기존 체크포인트를 추론 시점에 접는 소규모 llama.cpp 패치로 소비자 노트북에서 저장 절벽을 제거 — 미조정 스트리밍 기준선 대비 dense 70B 약 120배, 118B MoE 약 1.7배, 공격적 양자화 베이스라인과 병치, 그리고 57.6GB 체크포인트의 GPU 디코드 34.4 tokens/s — 하고, 프로덕션 루프 모델의 펼친 쌍둥이 실험: 상주만으로 15.1과 0.1 tokens/s가 갈리는 동일·함수 쌍의 시연

(§4.1~4.4). (5) 학습 없는 체크포인트를 접는 것이 얼마만큼의 가치가 있는지에 대한 실측 천장. 표적 복구는 접힌 118B MoE를 퍼플렉시티 4.2×10^4 에서 27.6까지 내리지만 (접지 않으면 6.40) 그 이상은 아니다: 오라클 라우팅 하한 아래에서 물리층의 256-전문가 집합 전체가 목표 잔차의 0.93~0.95를 남기며, 우연 바닥은 0.957이다. 접기가 만드는 궤적 위에서 그 전문가 기저는 이웃 층의 라우팅 함수에 대해 같은 차원의 임의 부분공간보다 근소하게 나을 뿐이다; 무접기 활성화로 풀면 스택은 0.74~0.95 구간이다. 따라서 실행된 접기의 게이팅 쪽에 한정된 복구에는 쓸 재료가 거의 없고, 이 시스템 기법은 루프 네이티브 가중치를 요구한다 (§4.5).

범위. 품질 결론은 TinyStories 데이터의 15M 규모(fp16 30.4MB)에서만 주장한다; 루프를 전제로 학습되지 않은 체크포인트를 접으면 출력 품질은 설계상 파괴되며, 우리의 접기 실험은 처리량 물리를 측정한다; §4.5가 학습 없는 복구가 품질을 어디까지 움직이고 어디서 멈추는지를 측정한다. 두 반쪽은 합쳐진다: 루프 학습 모델이 품질을 공급하고 (§3, [6,16]), 접기는 그런 모델을 소비자 하드웨어에서 매력적으로 만드는 상주 산술을 공급한다 (§4). 펼친 쌍둥이 실험 (§4.4)이 두 반쪽 사이의 간극을 닫는다 — 일관된 출력, 루프 네이티브, RAM 초과를 단일 측정 안에서.

2 관련 연구

순환 깊이. 층 재사용은 Universal Transformers[1]와 ALBERT로 거슬러 올라간다. MobileLLM[2]은 서브-비리언 온디바이스 모델에 인접 블록 공유를 채택했고, Relaxed Recursive Transformers[3]는 사전학습 모델을 루프별 LoRA와 함께 재귀형으로 변환하며, Huginn[4]은 깊이 64까지 루프해 테스트타임 컴퓨트를 확장하고, Mixture-of-Recursions[5]는 토큰별 재귀 깊이를 라우팅하며, Ouro[6]는 7.7T 토큰으로 루프 모델을 사전학습해 $1.4B \times 4$ 가 dense 4B와 대등함을 보였다. MELT[7]은 루프 간 KV를 공유한다. 루프형 MoE 자체도 확립되어 있다: MoEUT[18]은 공유층 (Universal Transformer) 형태에 FFN-어텐션 모두 MoE를 결합했고, LoopMoE[19]는 반복-조건 변조와 파라미터-FLOP 일치 루프-대-일반 MoE 비교를 더했으며, Hyperloop Transformers[20]는 middle 블록만 루프하고 — 우리 프로토퀴와 같은 begin/middle/end 분할 (§3.1) — 그 이점이 사후 양자화에서도 유지됨을 보고한다 (파라미터 일치 비교 축으로, 우리의 메모리-2배 회복 지표와는 다른 축이다); LT2[21]는 루프 내부의 어텐션을 선형 변형으로 대체한다. 이 계보 전체에서 명시된 이득은 파라미터, 품질, 또는 추론력이며, 저장 I/O는 측정되지 않는다.

Tied experts. 우리의 §3.3에 가장 가까운 Tying the Loop[8]은 연속 층에 걸쳐 expert FFN 가중치를 공유하되 층별 독립 라우터를 사용하며, attention을 풀어주면 tied 그룹 내 완전 재라우팅이 거의 무비용임을 보였다 — 우리의 re-route 조건과 일치하는 결과다. 그러나 이 연구는 라우팅을 루프 간 고정하는 것을 검토하지 않고, expert 접착 수나 대역폭도 측정하지 않으며, 프레이밍은 층 파라미터 절감이다. 우

리의 sticky 대 re-route 비교와 접근 지역성 결과가 그 공백을 차지한다.

저장 계층 추론. LLM-in-a-flash[9], AirLLM류 층 스트리밍, FlexGen[10]은 비루프 모델의 가중치를 작은 메모리로 불러보낸다; MoE 오프로딩 시스템[11,12]은 라우팅 지역성을 활용하지만, 우리 측정에 따르면 기본 모델은 그런 지역성을 신뢰할 만큼 제공하지 않는다. 어느 것도 스트리밍과 층 재사용을 결합하지 않는다. SHARP[13]은 인접 층 쌍을 공유해 iPhone에서 저장 용량과 로딩 시간 42% 절감을 보고한다 — 메커니즘상 인접하지만 루프가 아닌 쌍 공유이고, 그 모델들은 RAM에 들어가므로 RAM-초과 절벽 자체가 생점이 아니다. 2024년의 llama.cpp 논의[14]는 런타임 층 재사용을 제안했으나 구현 측정되지 않았다. Nanbeige4.2-3B[16]은 우리가 아는 한 최초의 프로덕션 루프 릴리스(22층×2)이며 파라미터-없는-용량으로 프레이밍된다; 우리는 이를 생태계 검증에 사용한다.

따라서 우리의 주장은 좁고, 다각도 조사(시스템 학회, [2~6, 18~21]의 인용 그래프, 특허, 커뮤니티)의 범위 안에서 비어 있다: 루프 × RAM 초과 × 저장 I/O 실측의 3중 결합. 루프 트랜스포머 영역이 이제 불빈다는 사실 — 공유층 MoE[18,19], 부분 루프 구조[20], 루프 효율 변형[7,21] — 은 이 관찰을 약화시키기보다 날카롭게 한다: 이들 중 누구도 저장 I/O, RAM-초과 디코드 처리량, 루프 내 라우팅 정책 비교를 보고하지 않는다. 이 문장의 프레이밍 위험을 명시적으로 표기해 둔다: 좁은 조건 셋의 교집합은 구성상 자명하게 비어 있으므로, 그 공백 자체는 기여의 증거가 아니다. 무엇이 새롭고 무엇이 아닌지 정확히 하면 — 우리는 새 메커니즘이 아니라 측정과 프레이밍의 기여를 주장한다. 층 재사용, 루프 간 KV 공유[7], 저장 계층 스트리밍[9~13]은 각각 개별적으로 확립되어 있다; 우리가 더하는 것은 (i) 실제 체크포인트에서 루프 하의 RAM-초과 디코드 절벽에 대한 최초의 직접 계측 — 동일-함수 대조 실험 포함 — (§4), (ii) 루프형 MoE 내부의 미측정 설계 축 (§3.3), (iii) 이 결합이 함의하는 상주 대 컨텍스트 트레이드오프 (§5, 그림 4)다. "3중 교집합이 비어 있었다"를 불충분한 근거로 여기는 독자는 (i)~(iii)을 대신 저울에 올리면 된다.

3 통제 품질 실험

3.1 프로토콜

단일 GPT 구현이 설정만으로 모든 변형을 표현한다 (middle-cycle 공유: 첫/끝 층 비공유 — Hyperloop Transformers[20]도 쓰는 begin/middle/end 분할; 입력 재주입; 루프 인덱스 임베딩; pre-norm RMSNorm). 데이터는 4k BPE 어휘의 TinyStories(학습 4.807억 / 검증 480만 토큰); 학습은 AdamW(6e-4, cosine), 유효 배치 32,768 토큰, 고정 시드로 모든 조건이 동일한 배치 순서를 소비한다. 지표: 검증 손실; MoE는 추가로 (토큰, 층-슬롯)당 접착한 유니크 expert 수의 평균(루프 누적). kill-gate는 실행 전에 고정했다. 회복률 = (small - loop)/(small - large): 상주 메모리를 2배로 늘렸을 때 얻는 품질 중 루프가 공짜로 얻는 비율로 정의한다. §3 전체에서 상주 동일성은 가중치 기준이다; KV는 논리 깊이에 비례해 커지며 §5에서 따로 계산한다.

3.2 Dense: 동일 상주 비교

각 2억 토큰으로 세 모델을 학습했다: *small*(4층, 깊이 4, fp16 상주 30.4MB), *loop*(같은 4층의 중간 블록을 $\times 3$ 루프, 깊이 8, 30.4MB), *large*(8층, 깊이 8, 56.1MB). 표 1이 보이듯 루프는 동일 상주 기준선을 명확히 이기고 2배 격차의 약 41%를 회복하며, 실험 전체가 독립된 하드웨어·정밀도에서 재현되었다. 민감도 대조는 통과했고(*large*-*small* = 0.091 nats), 루프 학습은 기준선 대비 1.9배의 시간이 들었다 — 연산으로 지불하는 예상된 대가다. §3의 회복률은 전부 단일 시드다; §3.3에서 측정한 시드 간 손실 분산 (± 0.0035 nats)을 0.091 nats 분모에 대입하면 개별 회복률 수치에 약 $\pm 4\%$ 의 불확실성이 함의된다 — §3.4의 후반 R 스윙 증분도 이 대역 안에 든다.

그 대가는 진짜 교란 변수다: 루프 조건은 동일-토큰 기준선보다 학습 FLOPs를 더 쓰므로, 회복된 격차의 일부는 "루프 자체"가 아니라 "더 쓴 연산"의 효과일 수 있다. 우리는 연산량 일치 대조를 실행했다: *small* 아키텍처를 루프 조건의 근사 FLOP 예산(4억 토큰, 약 2배)까지 처음부터 학습하면 검증 손실 **1.4936**에 도달한다 — *loop*(1.5577)는 물론 2억 토큰의 *large*(1.5038)보다도 낮다. 이 규모와 데이터 체제에서, 데이터가 아니라 연산이 병목일 때는 그 연산을 상주-크기 모델의 추가 토큰에 쓰는 편이 루프보다 낮다. 따라서 본 절의 품질 주장은 동일-토큰 결과다: 루프는 토큰 예산이 고정된 상황(데이터 제한·에폭 제한)에서 남은 연산을 회복 품질로 바꾼다. 분명히 적어 둔다: 우리 설정에서 이 체제는 내재적이지 않고 부과된 것이다 — 학습 토큰 4.807억이 있었고 2억 예산은 설계 선택이었다 — 따라서 동일-토큰 체제의 실용적 관련성은 데이터가 실제로 병목인 규모에 달려 있으며, 문헌의 대규모 루프 결과[6]는 우리 예산을 훨씬 넘겨 학습한다. 이 교란은 학습 측에만 있다: 디코드 시점에는 본 논문이 겨냥하는 대역폭·병목 하드웨어에서 추가 루프 FLOPs가 거의 공짜다 — §4의 실측(스트리밍 0.02 tokens/s, 토큰당 약 44GB의 SSD 읽기로 포화, 연산 아님)이 디코드 FLOPs는 교환 대상 병목이 아님을 보이므로, 상주/처리량 주장(§4)은 영향받지 않는다.

표 1. Dense 동일 상주 가중치 실험 (검증 손실, 2억 토큰). 두 열은 같은 설정·같은 시드(1337)·같은 토큰화 데이터 파일 — 따라서 같은 배치 순서 — 를 독립된 하드웨어/정밀도 스택에서 실행한 것이다; 이는 구현·하드웨어 재현이지 통계적으로 독립인 반복이 아니다(시드 분산은 §3.3에서 따로 정량화한다). 데이터 경로가 공유되므로 소수 3~4자리 일치는 기대되는 결과이며, *small*의 소수 4자리 정확 일치는 공표 자릿수에서의 반올림이지 복사된 수치가 아니다.

모델	상주	깊이	M1 Max fp32	Colab T4 fp16
<i>small</i>	30.4 MB	4	1.5950	1.5950
loop (R=3)	30.4 MB	8	1.5577	1.5573
<i>large</i>	56.1 MB	8	1.5038	1.5045

3.3 MoE: 루프 내 라우팅 정책

실제 대형 모델은 MoE이며, MoE를 루프시키는 것은 대역폭에 직결되는 질문을 낳는다: 블록이 다시 실행될 때 라우터가 expert를 다시 고르는가(re-route), 첫 통과와 선택을 재사용하는가(sticky)? 재라우팅은 토큰당 expert 합집합 — 우리의 122B 측정에서 캐싱을 죽인 바로 그 양 — 을 키울

수만 있는 반면, sticky는 이를 top-k에 동결한다. 42M 파라미터 MoE 4종(16 expert, top-2, d=384, 1억 토큰)을 학습했다: 무루프 기준선, R=3의 두 정책, 그리고 2배 메모리 상한.

표 2. 동일 상주 가중치의 루프형 MoE (최저 검증 손실; 접축 = (토큰, 층·슬롯)당 유니크 expert 평균, 루프 누적. MoE 모듈 4개 전체의 평균이며, 공유하지 않는 pre/post 블록은 1회만 실행되어 top-k=2가 상한이다 — 따라서 이 열의 천장은 6이 아니라 4다). 네 행 모두 Colab(T4/fp16) 스택이며 표 1의 둘째 열과 같다; 저장소에는 같은 구성의 M1/fp32 로컬 런 둘이 다른 사이클로 함께 실행 있으나(moe-small 최소 검증손실 1.7611, moe-loop-st는 step 2500에서 잘림) 이 행들과 비교 가능하지 않고 여기 수치의 출처도 아니다.

모델	검증 손실	접축	회복률
moe-small (무루프)	1.7655	2.000	—
moe-loop, sticky	1.7129	2.000	49.7%
moe-loop, re-route	1.7081	2.369	54.2%
moe-large (2 $\times$)	1.6596	2.000	100%

발견은 셋이다(그림 3). 첫째, 루프는 MoE에서 dense 못지 않게 — 오히려 더 잘 — 작동한다(회복률 49.7~54.2% 대 41%). 둘째, 정책 비교. 표 2의 단일 시드 런에서 sticky는 손실에서 re-route에 0.28% 뒤지지만(사전 등록 폐기 임계값: 2%), 단일 시드의 결정론은 통계적 강건성이 아니므로 n=3 시드로 비교를 반복했다: sticky-re-route 차이는 **+0.0032 $\pm$ 0.0035 nats** — 통계적 동물이다. 초고는 표 2를 "sticky가 re-route 이득의 92%를 유지한다"로 읽었으나, 그 프레이밍이 정량화한 격차 자체가 시드 잡음 안에 있으므로 이를 폐기하고, 더 단순하고 강한 진술로 대체한다: **n=3에서 루프 간 라우팅 자유는 측정 가능한 어떤 것도 사주지 않으므로, 대역폭·최적 정책 — 접축을 top-k에 동결한 sticky — 는 공짜다.** 함께 밝혀야 할 교란이 하나 있다: 부하균형 보조손실이 sticky 재사용이 건너뛰는 바로 그 분기에서 계산되므로, sticky 팔은 forward당 루프 모듈마다 보조항 하나를, 재라우팅 팔은 둘을 누적한다. 두 팔은 라우팅뿐 아니라 균형 정규화에서도 다르며, 따라서 이 비교는 — 아래 접축 수치를 포함해 — 라우팅 자유도 자체가 아니라 구현된 두 정책의 비교다. 루프 대 기준선 격차(약 0.055 nats)는 시드 잡음보다 한 자릿수 크며 유지된다. 셋째, 자유 재라우팅조차 루프 블록 안에서 가능한 6개 중 2.74개만 접축한다(표 2의 2.369는 MoE 모듈 4개 평균이고 그중 둘은 1회만 실행되어 top-k=2가 상한이다. 루프 블록만의 값은 $(4 \times 2.369 - 4)/2$: 학습된 라우터는 루프 간 약 81% 자기 일관적이며, 이는 라우터를 완전히 제약해도 감지 가능한 비용이 없는 이유를 시사한다. [8]과 정합적이다 — 그들의 층별 독립 라우터는 우리 re-route 조건의 유사물이다. 루프형·공유 expert MoE 구조는 존재하지만[8,18,19] 루프 내 라우팅 정책을 비교하거나 expert 접축을 계속한 것은 없다 — sticky 축과 접축 지표는 우리가 아는 한 여기서 처음 측정되었다. 용어 주의: [12]의 *sticky*는 토큰 축 라우팅 일관성(인접 토큰 간 expert 전환을 벌하는 학습 손실)을 뜻하고, 우리의 sticky는 추론 시 루프 축 일관성이다. 두 축은 직교하며 합성 가능하다.

3.4 R은 어디까지 가는가

$R \in \{1, 2, 3, 4, 6\}$ 을 각 2억 토큰으로 스윙했다(그림 1): 회복률은 $0 \rightarrow 27.6 \rightarrow 41.7 \rightarrow 48.5 \rightarrow 51.7\%$ 로 오르되 단계당 증분이 대략 반감한다(27.6, 14.1, 6.8, 이후 1.6/단계). 수확은 약 53~55%를 향해 포화한다; 로그 성장 가설은 기각되며 실용 최적점은 $R=3 \sim 4$ 다($R=6$ 은 연산 40%를 더 내고 +3.2점을 얻는다). 이 천장은 용량 직관과 맞는다: 루프는 순차 깊이를 살 뿐 파라미터 저장을 사지 않으며, 지식 용량은 유일 가중치의 것으로 남는다[15]. 격차의 남은 절반에 닿으려면 지식 축(더 많은 유일 파라미터, 검색, 메모리층)이 필요하다 — 루프와 그 기법들은 경쟁이 아니라 결합의 관계다. (§3.2의 연산량 일치 단서는 스윙 전체에 적용된다: 이들은 동일-토큰 비교다.)

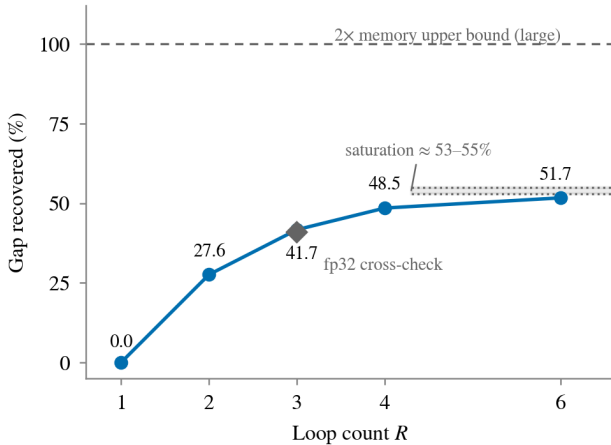


그림 1. 루프 수 R에 따른 2배-메모리 품질 격차 회복 비율 (dense, 15M 규모, 런당 2억 토큰). 증분이 단계마다 반감하며 53~55% 근처로 포화한다. R=3 점은 독립 fp32 스택에서 교차 확인(다이아몬드).

4 시스템: 저장 절벽의 제거

4.1 llama.cpp 그래프 접기 하네스

상주 주장을 실제 체크포인트에서 측정하기 위해 llama.cpp를 패치했다(두 아키텍처에 걸쳐 약 300줄): $LLAMA_FOLD_K=K$ 이면 빌드 루프는 논리 깊이를 유지하되 각 층의 가중치를 물리층 $il \bmod K$ 에서 가져온다. KV 캐시 슬롯·위치·콜백은 논리층을 유지하고, 가중치 결합 구조(헤드 수, dense 대 MoE 분기)는 물리층을 따르며, 슬라이딩 윈도우 위상 정렬은 assert로 강제한다(K는 SWA 주기의 배수여야 함). 파일에서 앞 K개 층의 페이지만 닿으므로 지연 mmap 하에서 상주 집합이 K/n_{layers} 배로 줄어든다 — GGUF나 로더는 건드리지 않는다. 전체 상주 예산은 접기가 줄여주지 않는 KV 캐시를 포함해야 한다(KV는 논리 깊이에 비례한다): 아래 70B 접기의 경우 가중치 21.98GB — 블록 밖 텐서(토큰 임베딩과 출력 투영, 1.45GB)는 접히지 않으므로 접힌 가중치는 파일 크기/R이 아니다 — + KV 약 1.3GB(ctx 4096, fp16) + 버퍼 약 1GB $\approx$ 24.3GB 대 가용 약 24~26GB: 접기 배수는 이 총합이 단히도록 선택하는데, R=2에서는 그 범위의 상단에서야 닫힌다. macOS에서의 측정 위생은 만만치 않았다(--no-repack 필수; 로드 프리 패치 비활성; 위치독과 스왑-델타 가드); 실패 양상은 부록 A에 기록한다.

§4.5는 여기에 더해 전문가 전용 접기 ($LLAMA_FOLD_EXPERTS=R$)를 쓴다: 믹서 — 어텐션 또는 게이트 델타넷과 그 노름·KV 투영 — 는 자기 논리층의 가중치를 유지하고, FFN 블록(라우터, 라우팅 전문가, 공유 전문가, 어텐션 후 노름)만 물리층에서 가져온다. 전문가가 Laguna 57.6GB 중 53.3GB이므로 이것만으로도 바이트의 92.5%가 접히면서 시퀀스 혼합 경로는 손대지 않는다. 또한 순환 사상과 달리 층별 예외를 허용한다 — 개별 층을 자기 자신에 고정할 수 있고, §4.5는 이것이 필요함을 보인다. 거기서 절제 실험으로만 쓰는 두 개의 추가 손잡이는 공유 전문가와 FFN 사전 노름을 논리층에 두는 것, 그리고 반복 층의 라우팅 분기에 상수 γ 를 곱하는 것이다.

4.2 결과: 절벽의 실측과 제거

표 3. M1 Max 32GB에서 기존 체크포인트 접기 (CPU 8스레드; 런당 생성 32토큰, 단 Laguna 스트리밍 기준선은 8토큰; 접기 행의 품질은 §4.5에서 별도로 측정함 — 루프 학습이 없는 체크포인트다). SSD 바이트는 런 전체의 페이지인 델타. 접기 행은 $n=3$ 반복의 평균, $\pm$ 는 표본표준편차. 가중치에 결합된 구조는 물리층을 따르므로 (§4.1), $K=16$ 의 Laguna 접기는 논리층 16과 32를 물리층 0에 사상한다. 물리층 0의 FFN은 선행 dense 블록이므로 ($n_layer_dense_lead = 1$, $ff = 12288$) 48층 중 2층이 256-전문가 MoE 블록 대신 dense FFN을 실행한다. 상주 바이트와 처리량은 영향받지 않지만, 접기 행의 출력 붕괴를 루프만의 탓으로 돌릴 수는 없다. 마지막 행은 바로 위 행의 백엔드 교체가 아니다: 세 층을 고정하고 공유 전문가를 논리층에 둔 전문가 전용 접기를 쓰므로 (§4.1, §4.5) 유일 가중치 집합이 그만큼 크다 (25.0GB 대 18.7GB, 둘 다 GGUF에서 텐서별로 합산). 상주 열은 RSS가 아니라 wired GPU 메모리다. SSD 열을 비운 것은 이 행의 1회성 로드 페이지인을 계측하지 않았기 때문이며, 가중치가 장치 전용 버퍼에 한 번 복사된 뒤 다시 읽히지 않으므로 스트리밍 행들의 토큰당 값과 같은 양이 아니다. 처리량은 단일 llama-bench 프로세스 내 3회 반복의 평균과 표본표준편차이며, 두 번의 퍼플렉시티 평가 직후 $-p 0 -n 32$ (거의 빈 KV 캐시)에서 측정했다. CPU 행들의 $n=3$ 은 별도 프로세스 기동에 런별 페이지인 회계를 붙인 것이므로 두 $\pm$ 는 같은 양이 아니며, Metal 행은 부록 A(6)이 말하는 기동 간 페이지 캐시 변동을 포집하지 않는다. 이 행은 또한 디코딩 경로만 측정한 것이다: 이 구성의 배치 프리퀀시는 NaN을 낸다 (§4.2, 부록 A(11)). 접기 사상·유일 가중치 집합·복구 손잡이·백엔드·KV 점유가 모두 다르므로 34.41과 2.37은 같은 축이 아니며 백엔드 가속으로 읽어서는 안 된다.

구성	Peak RSS	SSD 기	읽 스 합	디코딩 t/s
Llama-3.3-70B Q4 (42.5GB), 스트리밍	21 GB	1298 GB	0	∞ 0.02
접기 R=2 (유일 21.98GB)	21 GB	31~33 GB	0	2.40 $\pm$ 0.10
IQ2_XXS (17.8GB), 스트리밍, 일관 출력	20 GB	30 GB	0	1.7
Laguna-S-2.1 118B-A8B (57.6GB), 스트리밍	22 GB	37 GB	0	1.4
접기 R=3, K=16 (유일 18.7GB)	15~20 GB	18~20 GB	0	2.37 $\pm$ 0.42
접기 R=3 + 복구, Metal (유일 25.0GB)	25.2 GB wired	—	0	34.41 $\pm$ 0.74

dense 절벽은 가혹하고 그 제거는 극적이다(그림 2): 스트리밍 시 70B 모델은 토큰마다 활성 가중치를 거의 전부 다시 읽고 — 8토큰 런에서 355GB(토큰당 44GB), 32토큰 런에서 1298GB(토큰당 41GB; 42.5GB 파일의 31배) — 약 0.02 t/s로 디코딩한다(0.019: 32토큰에 벽시계 1713초); 유

일 가중치가 RAM에 들어가도록 접으면 콜드 읽기는 한 번 (31~33GB)뿐이고 디코딩은 2.40 ± 0.10 t/s로 돈다 — 환경 변수 하나로 약 120배, 스왑 0. 여기의 스트리밍 기준선은 최적화된 리더가 아니라 미조정 lazy-mmap 스트리밍이다: 순차 대역폭(약 2GB/s)은 이상적 이중버퍼 스트리머를 이 파일에서 약 0.047 t/s로 상한하므로, 가장 강한 스트리밍 기준선 대비 배수는 대략 50배가 된다 — 절벽은 살아남지만, 측정된 깊이는 기준선 의존적이다. MoE 절벽은 본질적으로 완만하므로(토큰당 활성 expert만 읽는다) 접기 이득은 약 1.7배다; 거기서의 이득은 활성 풀의 완전 상주다. (초기 단발 런은 3.8 t/s로 읽혔으나, 반복 측정이 그 초과분을 페이지 캐시 상태로 귀속시킨다 — 부록 A(6) — 우리는 $n=3$ 평균을 보고한다.) CPU 행들은 백엔드를 의도적으로 고정된 것이며 바닥이지 천장이 아니다. v2.2에서 Metal 경로는 미해결 공학 과제였다: 접힌 모델을 오프로드하면 매핑된 텐서 전부가 wired GPU 메모리로 등록되어 물리 RAM을 초과했다(부록 A(4)). 두 가지 변경이 이를 닫는다 — 접힌 그래프가 결코 읽지 않는 텐서는 로더가 아예 만들지 않고, 남은 mmap 기반 가중치는 매핑 전체를 wired로 잡는 대신 텐서 단위로 장치 전용 버퍼에 복사한다 — 그 결과 접힌 Laguna가 wired 25.2GB로 34.4 ± 0.7 t/s($n=3$)에 디코딩하며, 그것도 올바르게 한다. 배치를 맞춰 같은 그래프를 채점하면 Metal 27.17 대 CPU 27.27 — 백엔드 차이는 0.4%다. 다른 곳에서 보고한 27.59와의 나머지 간격은 백엔드가 아니라 배치다(CPU 값 자체가 ubatch 512에서 27.59, ubatch 1에서 27.27로 움직인다). 따라서 이 처리량은 빠른 쓰레기가 아니라 실제 계산이다. 57.6GB라서 이 기계에 애초에 상주할 수 없는 118B-A8B 체크포인트가 그 GPU 위에서 디코딩한다. 이 행에 대해서는 절벽 배수를 보고하지 않는다 — 스트리밍 기준선을 Metal에서 다시 재지 않았기 때문이며, 따라서 이 절의 모든 배수는 CPU 배수로 남는다. 한계 하나가 주장을 디코드 경로로 한정한다: 이 구성의 배치 프리필은 NaN을 낸다. ubatch 512에서 접기 배수를 훑으면 실패가 상주 크기 순으로 늘어선다 — $R=6 \cdot R=5 \cdot R=4$ (가중치 15.0~17.6GB)는 유한, 공유 전문가 손잡이 없는 $R=3$ (22.0GB, 퍼플렉시티 3.8×10^4)도 유한, 그 손잡이를 켜 $R=3$ (22.4GB)은 NaN, 실제 배포 구성(25.0GB)도 NaN, $R=2$ (31.3GB)는 명시적 `kIOGPUCommandBufferCallbackErrorOutOfMemory`. 그러면서 이들 전부가 배치 1에서는 정확히 디코딩한다. 이 기기가 보고하는 권장 최대 작업집합은 26.8GB이고, 그것이 실제 구속 조건이라는 가장 날카로운 증거는 22.0GB/22.4GB 쌍이다: 가중치 0.4GB 차이가 유한을 NaN으로 뒤집는데, 커널 결합이라면 그러지 않는다. 그럼에도 우리는 이것을 진단이 아니라 순서로 보고한다. `iogpu.wired_limit_mb`는 울리지 않았다. 이 프리필이 들어갈 만큼 올리면 시스템의 나머지가 돌아갈 몫이 남지 않기 때문인데, 그것이 곧 요점이다: 32GB 기기에서 작업집합 한도는 조정해 없앨 보수적 기본값이 아니라 배포 천장이다. 즉 접기는 애초에 상주할 수 없던 모델의 GPU 디코드를 사주고, 구속 조건을 SSD 대역폭에서 그 천장으로 옮긴다. 이 런들이 여전히 보여주지 않는 것은 품질이다: 접힌 출력은 열화되며, 이는 루프 학습이 없는 체크포인트에

서 예상된 바다. §4.5는 학습 없이 그것을 어디까지 복구할 수 있는지 측정하고, 남은 격차가 공학의 부족이 아니라 표현력의 문제임을 보인다 — 품질은 루프 네이티브 학습에서 와야 한다(§3, [3,6,16]).

표 3에 대한 회의적 독해는 이렇게 묻는다: 공격적 양자화만으로 70B 체크포인트를 예산 근처로 줄일 수 있는데 접기가 왜 필요한가. 우리는 그 베이스라인을 직접 실행했다: IQ2_XXS(가중치당 약 2.06비트, 17.8GB)는 평범한 SSD 트래픽(30GB)으로 1.7 t/s를 내며 출력은 일관적이다. 우리의 입장은 접기가 오늘의 기성 체크포인트에서 양자화를 이긴다는 것이 아니다 — 기성 체크포인트라면 공격적 양자화가 실용적 기본값이고, 그것만으로 가용 RAM에 도달하는 경우 접기는 더할 것이 없다. 접기의 가치는 전망적이며 직교적이다: (a) 루프 네이티브 모델(§3, §4.3~4.4)에는 약 2 비트급 품질세 없이 상주를 제공하며, 이는 비루프 모델의 사후 양자화가 같은 비트 수로 따라올 수 없는 것이다; (b) 양자화와 접기는 결합된다 — 접힌 모델 자체를 양자화할 수 있고($R \times Q$ 복리), 우리는 이 조합을 이미 답한 질문이 아니라 자연스러운 다음 측정으로 표기한다.

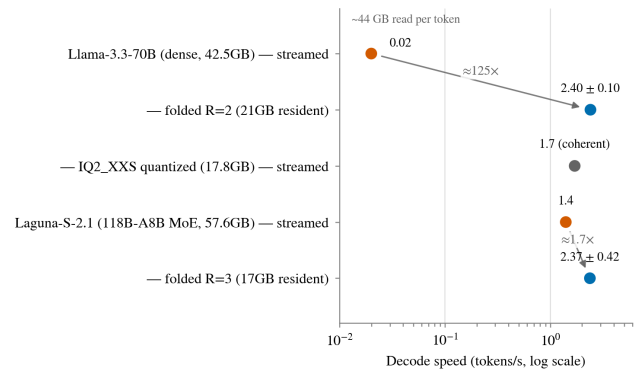


그림 2. 같은 노트북에서의 접기 전/후 디코드 속도 (로그 스케일). dense는 절벽 전체를 맞고(토큰당 약 44GB 재읽기) 약 120배를 읽으며, MoE는 활성만 읽으므로 약 1.7배를 얻는다. 공격적 양자화 베이스라인(IQ2_XXS, 회색)은 비루프 체크포인트의 실용적 선택지다: 접기보다 느리지만 출력이 일관적이다.

4.3 생태계 검증: 프로덕션 루프 모델

미학습 체크포인트 접기가 물리를 보였다면, 루프 네이티브 모델은 제품을 보인다. Nanbeige4.2-3B[16](22개 유일 층을 2회 실행; Q4에서 2.57GB)은 벤더의 llama.cpp 브랜치로 같은 노트북에서 동작한다: 출력은 완전히 일관적이며(추론 모드의 코드 답변) Metal에서 디코드 49.4 tokens/s / 프롭트 115 t/s를 기록한다. §3에서 15M 규모로 검증한 메커니즘은 이미 3B 규모로 출하되고 있으며, 그런 모델의 소비자 추론은 오늘 실용적이다; llama.cpp 메인라인 지원은 리뷰 중이다[17].

비교 가능성 주위: 49.4 t/s와 표 3의 2.4 t/s는 같은 축 위의 수치가 아니며 "여기서 루프가 20배 낫다"로 읽어서는 안 된다. Nanbeige는 약 3B, Q4, 루프 네이티브 체크포인트로 Metal 백엔드에서 RAM에 통째로 들어간다(2.57GB) — 그 수치는 절벽 제거가 아니라 평범한 소형 모델의 디코드 속도다. 표 3의 수치는 루프 학습이 전혀 없는 70B/118B 체크포인트를 순전히 저장 물리를 시험하려 사후에 접은 것으로, 의도적으로 고정된 CPU 백엔드다. 두 줄은 서로 다른 주장 — "루프 네이티브 모델은 이미 출하된다"와 "접기는 임의 체크포인트의 절벽을 제거한다" —

을 시연하며, 속도 순위표가 아니라 상보적 생태계 증거로서만 병치된다.

4.4 펼친 쌍둥이: 동일한 함수, 다른 상주

표 3에 대한 가장 강한 반론은 접힌 조건의 출력이 열화되어 속도와 품질이 한 번도 함께 시연되지 않는다는 것이다. 우리는 위의 프로덕션 모델로 만든 동일-함수 쌍으로 이를 단는다. Nanbeige4.2-3B는 22개 물리층을 2회 실행한다(논리 44층); 우리는 22층을 44층 비루프 GGUF로 복제한 **펼친 쌍둥이**를 구축하고(Q8_0: 집힘 4.43GB, 펼침 7.78GB), 모델의 루프 경계 정규화가 두 형태에서 같은 물리 위치에 적용되도록 추론 그래프를 패치했다. Nanbeige는 루프 경계마다 출력 norm을 재사용하는 RMSNorm을 삽입하며 이는 루프 수에 게이트되어 있다; 단순 펼치기는 이를 조용히 떨어뜨려 함수를 바꾼다. 우리의 패치는 이를 설정 가능한 물리 슬라이드에서 재현해 정확한 동치를 복원한다. 두 체크포인트는 같은 함수를 계산한다: 그리디 디코딩이 토큰 단위로 동일한 출력을 낸다. 제약이 없으면 둘 다 동일한 15.6 t/s로 디코딩된다(CPU 8스레드) — 이 쌍은 연산에서 전혀 다르지 않다. 이어서 mlock 뱀러스트로 19GB를 고정해 실측 8.1GB의 free-like 창을 남겼다: 접힌 형태는 들어가고 펼친 형태는 들어가지 않는다. 결과: 집힘 15.1 t/s(SSD 읽기 10GB); 펼침 0.1 t/s, 32토큰에 SSD 읽기 231GB, 전 구간 스왑 0 — 아키텍처·가중치·함수·출력·하드웨어가 모두 동일하고 오직 상주만 다른 **약 150배** 격차다. (§4.2와 같은 기준선 유보가 적용된다: 0.1 t/s는 lazy-mmap 스트리밍이다; 7.78GB 파일을 약 2GB/s로 읽는 이상적 순차 스트리머는 약 0.26 t/s가 상한이므로, 가장 강한 스트리밍 기준선 대비 격차는 약 60배다.) 이는 품질 교환을 구성상 제거한 표 3의 절벽이다: 일관된 출력의 루프 네이티브 RAM-초과 모델에서, 속도 차이는 상주 하나에 귀속된다.

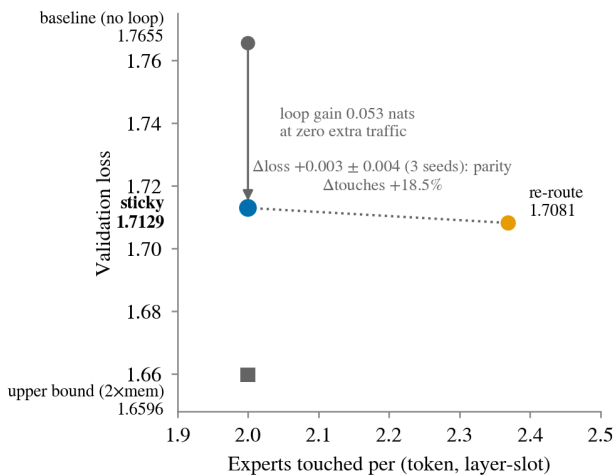


그림 3. 품질/트래픽 트레이드로 본 루프형 MoE 라우팅 정책 (§3.3). sticky는 기준선 트래픽(접촉 2,000)에 머물면서 자유 재라우팅과 통계적으로 구별 불가능한 품질을 낸다(3개 시드에서 $\Delta = +0.003 \pm 0.004$); 자유 재라우팅조차 가능한 4개 중 2,369개만 쓴다 — 네 MoE 모듈 전체 평균이고, 공유되지 않는 두 블록은 한 번만 돌아 top-k=2에서 막히기 때문이다. 루프 블록 자체만 보면 가능한 6개 중 2.74개다 (§3.3).

4.5 학습 없는 복구는 어디까지 가고, 어디서 멈추는가

표 3은 접기 행의 품질을 보고하지 않는다. 출력이 비일관적이기 때문이다. 그러나 그것은 한 구성에 대한 관찰이지

한계선이 아니며, 서로 다른 두 질문을 부른다: 손실 중 얼마가 **어떻게** 접었는지의 산물이고, 얼마가 공유되도록 학습된 적 없는 가중치를 접는 데서 오는 본질인가. 두 질문 모두 Laguna-S-2.1(48층, 라우팅 전문가가 256개와 공유 전문가가 1개, IQ4_XS)에서 답한다. 백엔드가 교란 변수가 되지 않도록 CPU 백엔드에서, wikitext-2의 앞 두 청크(1024 토큰)를 컨텍스트 512로 채점한다. 아래 모든 퍼플렉시티는 이 고정된 1024 토큰 창 위의 값이고 창에 따라 움직인다 — 4청크면 무접기 7.18, 최적 접기 36.96 — 따라서 서로 비교할 수는 있어도 다른 논문의 wikitext 수치와 비교할 수는 없다.

복구가 사는 것. 소박한 전문가 전용 접기 — 논리층 il 이 FFN 블록 전체를 물리층 wl 에서 가져오는 것 — 는 파국적이다: 퍼플렉시티가 접지 않은 6.40에서 $R=2$ 에 **1308**, $R=3$ 에 4.2×10^4 로 오른다. 세 가지 표적 변경이 그 대부분을 회수한다. (i) 공유 전문가는 논리층에 남아야 한다. 모든 토큰이 그것을 통과하므로, 접으면 라우터가 고르는 256분의 10 조각이 아니라 잔차 스트림 전체에 어긋난 변환이 걸린다. 이것 하나만으로 $R=2$ 가 1308에서 **21.8**로 내려간다. 이 복구는 상호작용이 강한 유일한 항목이고, 부호가 뒤집히므로 밝혀줄 값어치가 있다: $R=3$ 에서 단독으로 적용하면 파국적이며($4.2 \times 10^4 \rightarrow 5.6 \times 10^6$), 마지막 층들을 고정한 뒤에야 지배적 지렛대가 된다. 틀린 것 둘이 맞는 것 하나보다 나올 수 있고, 복구들을 따로 시험했다면 이 순서를 찾지 못했을 것이다. (ii) **마지막** 층들은 자기 자신에 고정해야 한다. $R=3$ 에서 층 47 하나만 고정해도 4배가 바뀌지만($4.2 \times 10^4 \rightarrow 1.0 \times 10^4$), 대신 앞쪽 두 층(1과 2)을 고정하면 아무것도 달라지지 않는다(3.8×10^4) — 용량 효과가 아니라 위치 효과이며, 마지막 블록이 연임베딩에 직결된다는 사실과 일치한다. (iii) 반복 층의 라우팅 분기를 감쇠시키는 스칼라 γ 는 $\gamma = 0.5$ 에서 내부 최적점을 가지며 46.5 $\rightarrow$ **27.6**의 추가 이득을 준다. γ 를 닫힌 형식 최소제곱으로 층별 적합해도 전역 상수를 이기지 못한다(36.4, 균일 재조정 후 27.7). 이는 남은 오차가 이득 불일치가 아니라는 첫 신호다. (i)에 대한 대조로 FFN 사전 노름을 논리층으로 옮기는 대칭적 개입도 시험했는데, 오히려 나쁘다(1465 대 1308). 두 정규화는 교환 가능하지 않으며, 공유 전문가가 결과는 "논리층에 더 많이 남기면 좋다"는 일반 효과가 아니다.

복구가 살 수 없는 것. 접지 않은 6.40 대비 27.6은 여전히 쓸 수 없고, 실패 양상은 정도의 문제가 아니라 질적인 것이다: 그리디 디코딩은 잘 형성된 문장 하나를 내고는 그것을 그대로 반복하며, 퍼플렉시티 46에서나 27.6에서나 동일하다. 지표와 사용성이 분리된 것이다. 추가 공학이 이 격차를 닫을 수 있는지 판정하기 위해 도달 가능성 천장을 측정했다. 논리층 il 이 wl 에 접혔다고 하자. 공유 전문가를 논리층에 두었으므로 라우팅 분기가 만들어야 할 목표는 $R = \text{FFN}_{il}(X) - \text{shexp}_{il}(X)$ 이다. expert_weights_norm 하에서 그 분기가 도달 가능한 집합은 물리층 wl 의 전문가 256개 출력의 스케일된 블록집일고, 이는 그들의 생성공간에 포함된다. 따라서 토큰마다 전문가 256개 전체에 대해 제약 없는 최소제곱을 풀면, 어떤 라우팅으로도 — 오라클 라우터든, 재적합된 라우터든, 게이팅 쪽 어댑터든 — 이길

수 없는 잔차의 엄밀한 하한이 나온다. 입력 X 는 접힌 모델 자신의 궤적에서 취했다. 이 선택은 중립적이지 않으며, 아래에서 양쪽을 모두 보고한다.

표 4. Laguna-S-2.1 전문가 전용 접기의 도달 가능성 천장. 값은 잔차 노름을 목표 노름으로 나눈 상대 잔차이며 낮을수록 도달 가능하다. 천장은 물리층 w_l 의 전문가 256개 전체에 대한 토큰별 제약 없는 최소 제공으로, 어떤 라우팅도 이길 수 없는 하한이다. 두 천장 열은 X 를 어느 궤적에서 취했는지만 다르다: 접힌 모델 자신의 활성화, 또는 무접기 모델의 활성화. 우연 바닥은 3072차원 속 임의의 256방향의 차원 계수만으로 달성하는 값 $\sqrt{(1-256/3072)}$ 이며, i.i.d. 가우시안 기저의 실측이 0.957이다. 256이라는 개수를 쓰는 것이 옳다: 토큰별 전문가 출력 행렬은 사실상 완전 랭크이며(조건수 3.2~7.7, 에너지의 99%를 담는 데 256 중 248~250개 특이방향이 필요), 따라서 전문가들이 저차원 부분공간으로 붕괴해 이 바닥이 지나치게 관대해지는 일은 없다 — 99% 랭크로 다시 계산하면 0.9574 대신 0.9584다. "검증"은 동일한 폴이를 층 il 자신의 전문가에 대해 반복한 것으로, 참해가 실현 가능하므로 잔차가 0이어야 한다. 소수 셋째 자리까지 0이 나온 것은 전문가 슬라이싱·역양자화·해법이 서로 정합적임을 검증하지만 외부 검증은 아니다 — 목표와 기저가 같은 순전파 코드에서 나오므로 둘이 공유하는 오류는 드러나지 않는다. 그 순전파는 llama.cpp 그래프 구성과 대조해 검사로 검증했다. T=128 토큰; 풀지 않고 실제 실행된 접기의 라우팅 분기 잔차는 1.0을 넘는다, 즉 아무것도 내지 않는 것보다 나쁘다.

논리층 il	물리층 w_l	천장, 접힌 X	천장, 무접기 X	검증 층 (il)
5	4	0.925	0.778	0.000
24	22	0.933	0.738	0.000
41	40	0.953	0.951	0.000
우연 바닥 (3072차원 속 임의의 256 방향)		0.957	0.957	—

바닥에 견주어 읽으면 결과는 날것의 숫자보다 좁고, 성립하는 곳에서는 더 날카롭다. 접힌 궤적에서 천장은 0.93~0.95이고 우연 바닥은 0.957이다: 전문가 집합 전체에 대한 오라클 라우팅은 라우팅 분기 노름의 30~37%, 에너지의 9~14%를 회수하며, 같은 차원의 임의 부분공간이 내는 8.3%와 견준다. 즉 실제 실행된 접기 위에서는 물리층 w_l 의 전문가 기저가 논리층 il 의 목표에 대해 임의의 256방향보다 근소하게 나올 뿐이다.

그러나 접힌 궤적 자체가 붕괴한 것이고, 그것이 중간층의 하한을 부풀린다. X 를 무접기 런에서 취해 같은 폴이를 반복하면 천장이 $il=5$ 에서 0.925 → 0.778로, $il=24$ 에서 0.933 → 0.738로 떨어진다 — 도달 가능한 에너지가 3.5배다 — 반면 $il=41$ 은 0.951에 머문다. 어떤 복구는 접힌 전 층에 동시에 적용되어 다음 층이 보는 궤적을 부분적으로 회복시키므로, 중간층에 대한 정직한 진술은 구간이다: 결합 복구의 천장은 0.74와 0.93 사이에 있고 우리는 비관적 끝만 측정했다. 우리는 처음에 이 구간을 중간층에 한정하고 깊은 층은 예외라고 보고했다. $il=41$ 이 두 궤적 모두에서 0.951이었기 때문이다. 나머지 깊은 접힌 층으로 폴이를 확장하면 그 주장은 반박된다: $il=42$ 는 — 한 층 더 깊고 같은 물리층 40을 공유하는데 — 무접기 궤적에서 0.810, $il=44$ 는 0.878이며, $il=41$ 의 0.951과 대비된다. $il=42$ 의 독립 널 둘(입력 서플, 먼 물리층)이 모두 우연 바닥에 떨어지므로 0.810은 해법의 산물이 아니라 실제 기저 일치다. 따라서 $il=41$ 은 대표가 아니라 우리가 측정한 가장 비관적인 층이고, 깊은

층 예외는 없다: 무접기 궤적에서 접힌 스택 전체가 0.74~0.95 구간이다.

재작성기로도 회수되지 않는다. 논리층 24에 대해 w_l 을 거리 1부터 16까지 훑으면 천장은 0.923에서 0.957로만 움직이며 거리에 단조롭다($w_l=25$ 에서 0.923, 23에서 0.928, 거리 2에서 0.933~0.935, 거리 4에서 0.945, 거리 12와 16에서 정확히 우연 바닥 0.957). 배포된 사상은 거리 1과 2를 쓰고, 스윙의 최선점 $w_l=25$ 는 인과적 접기 사상이 쓸 수 없는 뒤쪽 층이다; 쓸 수 있는 짝 중 최선은 0.928이다. 즉 여기의 사상은 R 을 고정된 재작성기 중에서는 이미 최선에 가깝다 — 이는 사상 탐색이 무용하다는 말과 다르다. R 을 바꾸거나 층을 고정하는 것은 분명히 통하기 때문이다 (§4.5(ii), 그리고 $R=2$ 는 21.8로 $R=3$ 의 27.6보다 낮다).

이 측정이 닫는 것은 한 계열이다: 입력을 고정된 채 게이팅 쪽에 한정된 복구 — 오라클 라우터, 재조합된 라우터, 게이팅 어댑터, 라우팅 분기에 걸리는 스칼라나 벡터 이득 — 는 접힌 층이 무접기 층의 라우팅 출력을 재현하게 만들 수 없다. 목표가 도달 가능 집합 거의 바깥에 있기 때문이다. 그러나 혼합 계수가 아니라 기저나 목표를 움직이는 복구는 단지 못한다: 항상 켜져 있는 공유 전문가 경로에 어댑터는 R 자체를 바꾸므로 이 틀 밖이고, 입력 쪽 변조는 전문가에 다른 x 를 먹이는데 SwiGLU가 비선형이라 생성 공간 논증이 덮지 못한다. 자연스러운 입력 쪽 선택 하나 — 두 층 FFN 노름의 비로 재조정 — 를 시험했고 아무것도 사지 못했다(0.9293 대 0.9292). 그러나 이는 음성 관측 하나이지 닫힌 문이 아니다.

한 가지 외삽은 반드시 거부해야 하며, 그것을 거부하는 것은 우리 자신의 데이터다: 이 하한은 게이팅 쪽 변경이 퍼플렉시티를 낮출 수 없다고 말하지 않는다. 하한이 다루는 층별 목적함수와 종단간 목적함수는 정작 문제가 되는 구간에서 반상관이다. γ 를 층별로 적합해 층별 잔차를 최소화하면 중앙값 0.15로 물리는데 그 설정의 종단간 점수는 40.8이고, 층별 최적에서 한참 떨어진 $\gamma = 0.5$ 가 27.6이다. 층별 재구성 오차를 낮추면 퍼플렉시티가 약 50% 나빠진다. 이는 층별 복구 일반에 대한 경고다. GPTQ/SparseGPT 계열이 층별 재구성을 목적함수로 삼기 때문인데, 여기서는 그것이 틀린 목적함수다.

살아남는 것은 불가능성 증명이 아니라 기저에 대한 기계론적 진술이다. 접기가 실제로 만들어내는 궤적 위에서 물리층 w_l 의 전문가 기저는 논리층 il 의 라우팅 함수를 담고 있지 않다 — 따라서 실행된 접기의 게이팅 쪽에 한정된 복구에는 쓸 재료가 거의 없다. 이는 루프 변환[3]의 가격이 보정 배치가 아니라 수백억 토큰으로 매겨지는 이유와 정합적이다: 그 절차는 충분한 기저 위에 보정을 없는 것이 아니라 공유 기저를 만드는 것이다. §4.2와 함께 읽으면 두 결과가 기여의 경계를 긋는다. 접기는 저장 상주 기법이며, 그 축에서는 이제 32GB 노트북에서 118B 체크포인트의 GPU 디코드를 34.4 t/s로 올린다. 이 측정은 접기가 루프 학습 없는 체크포인트에 대한 사후 압축 기법이 될 수 있으리라 기대할 근거를 주지 않는다.

하한의 적용 범위. 한 체크포인트, 세 개 층, 한 문서에서 연속으로 뽑은 128토큰, 전문가를 동결한 FFN 전용 접기. 최소제곱 완화 때문에 정확한 추정치 아니라 하한이다 — 실제 제약 최적해는 이보다 나올 수 없다. 이것이 한정하는 것은 입력을 고정한 게이팅 쪽 변경에 의한 층별 출력 재구성이다; 종단간 손실은 한정하지 않으며, 위의 γ 결과가 둘이 교환 가능하지 않음을 보인다. 전체 모델 학습이 도달하는 범위에 대해서는 아무 말도 하지 않으며, 층들이 전문가 기저를 공유하도록 학습된 아키텍처에 대해서도, 이런 방식으로 분석하지 않은 믹서 접기에 대해서도 마찬가지다. 일반적 결과로 읽기 전에 두 번째 루프형 MoE 체크포인트에서 재현되어야 한다.

5 논의와 한계

규모. 품질 결과는 TinyStories의 15M(dense)·42M(MoE) 파라미터에서 확립되었다 — dense 팔의 30.4MB는 파라미터 수가 아니라 상주 바이트 수치다; 0.1~8B의 문헌[2,3,6]과 3B의 Nanbeige가 이전 가능성을 지지하지만, 우리 수치는 그 규모 너머를 주장하지 않는다. 우리의 토큰 예산은 루프에 보수적이며(루프는 장기 학습에서 이득이 커진다고 보고된다[6]) 따라서 회복률은 과소추정일 수 있다. TinyStories는 반대 방향으로 작용한다: 지식 요구가 낮고 구문 규칙성이 높아 — 루프가 가장 잘하는 체제라 — 같은 수치가 데이터셋 특성 쪽으로는 과대평가일 수 있다; 두 방향의 순 방향은 측정하지 않았다. **지식 천장.** 격차 절반에서의 포화는 루프가 저장이 아닌 깊이를 산다는 것과 일치한다[15]; 이득은 추론성 능력에 집중되며 지식 중심 용도는 파라미터/검색 축이 필요하다. **KV 캐시와 컨텍스트 길이.** 접기는 상주 가중치를 줄이지만 KV 캐시는 줄이지 못한다 — KV는 R과 무관하게 논리 깊이와 컨텍스트 길이에 비례한다. 그림 4가 Llama-3.3-70B(80층, GQA kv-헤드 8, 헤드 차원 128; fp16에서 토큰당 320KiB)의 체제 지도를 명시한다: 가용 24GB 예산 아래, §4.1이 잡는 런타임 버퍼 약 1GB를 같이 세면 R=2(가중치 21.98GB)는 약 3k 토큰의 컨텍스트만 유지한다; R=3은 약 24k, R=4는 약 34k에 이르고, KV를 q8로 낮추면 R=2가 약 6k로 늘어난다. 따라서 루프간 KV 공유[7]는 선택적 다듬기가 아니다: 낮은 접기 배수(더 나은 회복 품질, §3.4)와 고정 RAM에서의 장문맥을 모두 원하는 어떤 배포에서도 성립 조건이다. 우리는 이를 평가하지 않았으며, §3의 품질 결과와 §4의 장문맥 배포 사이에 놓인 가장 큰 미해결 항목으로 표기한다. **시스템.** 측정은 한 기기(M1 Max, 한 종류의 NVMe)다. v2.2에서 열려 있던 Metal 경로는 이제 돈다(§4.2). 그러나 절벽 행들은 여전히 CPU 백엔드이고 스트리밍 기준선을 Metal에서 다시 재지 않았으므로 여기 보고된 모든 배수는 CPU 배수다. Metal 행은 또한 우리가 상류에 반영한 수정이 아니라 상류 스케줄러 결함에 대한 우회기대고 있으며(부록 A(8)), 디코드만 포함한다 — R=3의 배치 프리필은 이 기기의 작업집합 한도를 넘어 NaN을 낸다(§4.2, 부록 A(11)). 따라서 우리가 보인 것은 접힌 118B 체크포인트의 GPU 디코드이지 종단간 서빙 경로가 아니다. **접기 품질.** v2.2는 이를 선택으로 회생했고, v3는 측정으로 한정한다(§4.5). 한정 쪽이 더 유용한 진술인데, 값싼 복구 계열 전체를 이루는 대

신 배제하기 때문이다. 따라서 실전 배포는 루프 학습 또는 루프 변환[3] 체크포인트를 요구하며 — 그 변환 비용(수십 B 토큰)은 우리가 지불하지 않았다 — §4.5가 그 비용을 보정으로 피할 수 없는 이유에 대한 우리의 설명이다. 하한 자체는 단일 체크포인트에서 얻었고, 접힌 모델 자신의 붕괴한 제적 위에서 측정되어 중간층에 대해 부풀려져 있으며, 학습 없는 복구 일반이 아니라 게이팅 쪽 복구를 한정한다. **주장 규율.** 모든 kill-gate는 사전 고정되었고, 신규성 주장은 집필 전 적대적으로 수색되었으며 그 프레이밍 위험의 인정과 함께 §2에 서술된다. 본 개정판은 적대적 리뷰를 반영한다: 멀티시드 반복(§3.3), 연산량 일치 대조(§3.2), 양자화 베이스라인(§4.2), 펼친 쌍둥이(§4.4)는 각각 리뷰가 요구했기에 존재하며, 초고의 헤드라인 주장 둘("92%"와 MoE 2.7배)은 추가 측정에 의해 폐기되거나 정정되었다. v2.1에서는 독립 AI 리뷰 세션들(저자 각주 참조)의 사후 교차 검증이 원래 신규성 수색이 놓친 2024-2026 루프형 MoE 계보[18-21]와 §3.3의 토큰 축/루프 축 sticky 구분을 추가했다. v3 역시 4개 렌즈 적대적 리뷰를 거쳐 개정되었다. §4.5가 애초에 없던 우연 바닥과 두 번째 궤적을 보고하는 이유, 그리고 Metal 프리필 실패를 진단이 아니라 순서로 보고하는 이유가 그것이다; 리뷰는 초고가 자기 하한을 잘못 읽은 것도 반박했다(상대잔차 0.93은 목표 노름의 30~37%에 해당하며, 처음 쓴 5~7%가 아니다). 아울러 v3 초안이 정오표 없이 바꾼 v2.2 수치 셋을 여기 밝힌다: 접힌 70B 가중치 21.2 → 21.98GB(파일 크기/R이 아니라 텐서별 합산이며, 이로써 그림 4의 모든 컨텍스트 천장이 낮아진다 — R=2가 약 9k에서 약 6k로), 루프 블록 내부 전문가 접 축 2.37 → 2.74(표 2의 2.369는 네 모듈 평균이고, 약 81% 자기일관성과 정합적인 값은 2.74뿐이다), 표 1의 large 상주 가중치 55.9 → 56.1MB. §1의 라우터 커버리지 수치는 98%에서 공개 census가 뒷받침하는 75%로 정정했다.

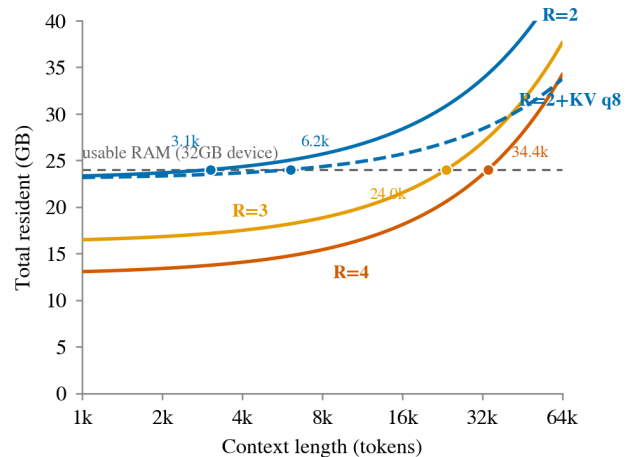


그림 4. Llama-3.3-70B의 R=2/3/4(fp16 KV)와 R=2+q8 KV에 대한 총 상주 메모리(접힌 가중치 + KV 캐시) 대 컨텍스트 길이. 마커는 32GB 기기의 가용 24GB 예산 아래 각 구성이 유지하는 최대 컨텍스트를 표시한다. 이 지도는 실측이 아니라 해석적 산식(접힌 가중치 + 토큰당 KV, 전부 집진 GB)이다. 접힌 가중치는 GGUF에서 텐서 단위로 합산했으며, 접히지 않는 블록 밖 텐서를 포함한다. 정확한 접기는 K가 층수의 약수여야 한다: 80층에서 R=2(K=40)와 R=4(K=20)는 실현 가능하고, R=3 곡선(K≈26.7)은 추세 표시용 이상화 보간이다. 표 3의 접기 행은 K=40(Illama, R=2)과 K=16(laguna, 48층, R=3)을 쓴다.

6 결론

상주 최적화로 다루면 순환 깊이는 접기 배수를 배포의 손잡이로 바꾼다: 수확이 아직 풍부한 $R=3\sim4$ 에서 학습하고, 접힌 스택으로 배포하고, 논리 깊이로 실행하며, 대역폭에 묶인 하드웨어에서는 sticky로 라우팅하라 — 멀티시드 데이터가 비용이 전혀 없음을 보인 정책이다. 32GB 노트북에서 이는 초당 백분의 몇 토큰짜리 저장 절벽 체제를 RAM 속도의 디코딩으로 바꾸고, 로더를 접힌 작업 집합에 맞추면 GPU까지 닿는다: 담을 수 없는 기계에서 57.6GB 체크포인트가 34.4 tokens/s로 디코딩한다. 펼친 쌍둥이 실험이 그 전환을 상주 하나에 귀속시킨다 — 같은 함수, 같은 출력, 약 150배의 속도.

접기 배수가 사주지 못하는 것은 공유되도록 학습된 적 없는 가중치 위의 품질이다. 학습 없이 그것을 어디까지 밀 수 있고 어디서 멈추는지를 측정했다: 접기가 만드는 제적 위에서 물리층의 전문가 기저는 이웃 층의 라우팅 함수에 대해 임의의 부분공간에 가깝고, 게이팅 쪽 변경으로는 그 너머에 닿지 못한다. 이것이 이 기법이 임의의 체크포인트를 위한 압축 트릭이 아니라 루프 네이티브·루프 변환 체크포인트를 위한 배포 기법인 이유이며 — 오늘 그 조건을 만족하는 것은 §4.3/§4.4 부류이지 물리를 분리하려 접었던 70B·118B 행이 아니다. 천장 결과는 접기 너머의 경고도 담는다: 층별 복귀 기법들이 최적화하는 층별 재구성 목적 함수가 여기서는 종단간 손실과 반상관이다. 품질 격차의 남은 절반은 루프를 지식 축 기법(메모리층, 검색)과 결합하는 방향을 가리키며, 이는 향후 과제로 남긴다.

† 문헌 조사, 구현, 실험 오케스트레이션과 야간 운영, 그리고 v3에서는 백엔드 스케줄러 결합의 진단(부록 A(8))과 도달 가능성 천장 측정의 설계 (§4.5)가 AI 연구 조수(Claude, Anthropic Fable 5 및 Opus 5)와 함께 수행되었으며, 이 개정판이 반영한 적대적 리뷰들도 포함한다: 연구 방향, 모든 결과의 채택 여부, 자원, 내용에 대한 책임은 인간 저자에게 있다. 모든 측정은 저자의 로그와 공개 저장소(github.com/cornch-k/fold-dont-stream)에서 재현 가능하다; 총 컴퓨트 비용은 GPU 크레딧 \$9.99 1회 구매와 소비자 노트북 한 대였다. 본 문서는 작업 중 프리프린트로 동료 심사를 받지 않았다. 본 한국어판은 영문판의 해석본이며 수치·주장에서 영문판이 우선한다.

참고문헌

- [1] M. Dehghani et al. Universal Transformers. arXiv:1807.03819, 2018.
- [2] Z. Liu et al. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. ICML 2024. arXiv:2402.14905.
- [3] S. Bae et al. Relaxed Recursive Transformers. arXiv:2410.20672, 2024.
- [4] J. Geiping et al. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. arXiv:2502.05171, 2025.
- [5] S. Bae et al. Mixture-of-Recursions. NeurIPS 2025. arXiv:2507.10524.
- [6] ByteDance. Ouro: Looped Language Models. arXiv:2510.25741, 2025.
- [7] V. Conchello Vendrell et al. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models (MELT). arXiv:2605.07721, 2026.
- [8] M. Jaggi. Tying the Loop — Tied Expert Layers in Mixture-of-Experts Language Models. arXiv:2606.16825, 2026.
- [9] K. Alizadeh et al. LLM in a Flash: Efficient LLM Inference with Limited Memory. arXiv:2312.11514, 2023.

- [10] Y. Sheng et al. FlexGen: High-Throughput Generative Inference of LLMs with a Single GPU. ICML 2023.
- [11] MoE-Infinity: Offloading-Efficient MoE Model Serving. arXiv:2401.14361, 2024.
- [12] A. Kayyam. Sticky Routing: Training MoE Models for Memory-Efficient Inference. arXiv:2607.08780, 2026; X. Zhu et al. ReMoE: Boosting Expert Reuse through Router Fine-Tuning in Memory-Constrained MoE LLM Inference. ICML 2026. arXiv:2605.27081.
- [13] SHARP: Sharing Adjacent Layers for Efficient On-Device Inference. arXiv:2502.07832, 2025.
- [14] llama.cpp discussion #4718: in-situ auto-frankenmerges (미구현 제안), 2024.
- [15] Z. Allen-Zhu, Y. Li. Physics of Language Models, Part 3.3: Knowledge Capacity Scaling Laws. arXiv:2404.05405, 2024.
- [16] Nanbeige. Nanbeige4.2-3B: a Looped-Transformer agent model. HuggingFace model card, 2026.
- [17] llama.cpp PR #25994: Add support for Nanbeige4.2 (looped transformer), 2026.
- [18] R. Csordás, K. Irie, J. Schmidhuber, C. Potts, C. D. Manning. MoEUT: Mixture-of-Experts Universal Transformers. NeurIPS 2024. arXiv:2405.16039.
- [19] W. Chen et al. LoopMoE: Unifying Iterative Computation with Mixture-of-Experts for Language Modeling. arXiv:2606.04438, 2026.
- [20] A. Zeitoun, L. Torroba-Hennigen, Y. Kim. Hyperloop Transformers. arXiv:2604.21254, 2026.
- [21] C. Deng et al. LT2: Linear-Time Looped Transformers. arXiv:2605.20670, 2026.

부록 A 소비자 하드웨어에서의 측정 위생

열한 가지 교훈을 다른 이들이 건너뛰도록 기록한다; 앞의 다섯은 하루 저녁을 소모한 실패 양상이다. (1) llama-bench는 ARM 가중치 repack을 끝 수 없다; 42.5GB 모델의 repack은 그에 상응하는 익명 메모리를 할당해 런을 OOM으로 죽인다 — llama-cli --no-repack을 쓰라. (2) 현행 llama-cli TUI는 대화 모드를 꺼도 -n 토큰 후 종료하지 않는다; 성능 라인을 감지해 종료시키는 위치독이 필요하다. (3) 기본 로드 프리페치는 파일 전체를 건드려 상주 관측을 파괴한다; 비활성화하라. (4) Apple Silicon에서 접힌 모델의 Metal 오프로드는 매핑된 텐서 전부를 wired GPU 메모리로 등록했다(32GB 기계에 42.5GB wired) — 프로세스 RSS가 1GB로 읽히는 동안 시스템이 얼어붙었고, RSS 스왑 기반 가드는 이를 보지 못한다. v3에서 해결했다(§4.2): 접힌 그래프가 읽지 않는 가중치는 텐서 생성 자체를 건너뛰고, 나머지는 텐서 단위로 장치 전용 버퍼에 복사한다. 그러면 wired 메모리가 접힌 작업 집합을 따라간다. 이 관측 불가 자체는 Metal을 넘어 일반화된다: mmap 기반 가중치 페이지는 파일 백드라서 RSS에도 스왑에도 계상되지 않으므로, 활성 상태 보기가 수백 MB를 보고하는 동안 프로세스가 수십 GB를 상주시킬 수 있다. macOS에서 정직한 상주 신호는 vm_stat 페이지인 회계뿐이며, 본 논문의 모든 SSD 읽기 열이 그것이다. (5) 스왑-델타 가드(+1.5GB 초과 시 중단)와 런별 페이지인 회계가 보고된 모든 런의 무스왑을 검증 가능하게 했다. (6) mmap 기반 모델의 단발 처리량은 신뢰할 수 없다: 초기 Laguna 접기가 3.8 t/s로 읽혔으나 $n=3$ 반복이 2.37 ± 0.42 로 정정했다 — OS 페이지 캐시는 런 사이의 숨은 변수다; 반복 평균을 보고하라. (7) 쌍

등이 실험(§4.4)에서 RAM 제약은 VM이나 cgroup이 아닌 mlock 벨러스트 프로세스로 걸었다; 무제약 대조(두 쌍둥이 모두 동일한 15.6 t/s)가 붕괴의 원인이 펼치기 자체가 아니라 제약임을 확인하는 검증을 겸한다. (8) ggml의 백엔드 스케줄러는 MoE 전문가 스택을 필요할 때 가속기로 복사하지만, 복사본을 그래프 분할별이 아니라 (텐서, 백엔드, 슬롯)으로 전역 메모이저한다. 앞선 분할이 이미 "복사한" 스택을 뒤의 분할이 참조하면 복사가 발행되지 않고, 커널은 초기화되지 않은 장치 전용 버퍼의 기록된 적 없는 바이트를 읽는다 — NaN 로깅, 오류 없음, 경고 없음. 접기 없이도 재현되며, 라우팅 *ids*에는 적용되었으나 가중치 피연산자에는 적용되지 않은 상류 수정과 같은 계열이다. op 오프로드 비활성화는 이 경로를 피해 CPU 값을 소수 넷째 자리까지 되돌린다(1307.5595) — 그러나 그 런들은 -ngl 0이라 이 스위치가 Metal을 계산에서 통째로 빼기도 하므로, 이 일치는 우회이지 메커니즘의 확인이 아니다. 확인 시험, 즉 op 오프로드를 켜 채 전체 텐서 복사를 강제하는 것은 돌리지 않았고, 코드에서 추론한 접기 비의존성을 실증할 독립 재현기도 작성하지 않았다. (9) 접힌 모델을 층별로 분석하려 활성화를 덤프할 때는 각 덤프를 가중치 텐서가 아니라 반드시 논리층 — 활성화 텐서 — 으로 키잉하라. 접힌 층들은 가중치를 공유하므로, 가중치로 키잉한 덤프는 여러 논리층을 조용히 한 파일에 합치고 이후의 모든 수치가 소리 없이 들어진다. 우리가 이를 잡은 것은 접히지

않은 층이 자기 자신과의 코사인을 0.449로 보고했기 때문인데, 이는 불가능한 값이다. 여기의 모든 비교 스크립트가 기댓값을 사전에 아는 자체 검증을 달고 있는 이유다. (10) atoi/atof로 파싱하는 환경변수 손잡이는 빈 문자열을 0으로 읽는다. 셀 인용 실수로 γ 가 빈 값으로 전달되어 $\gamma=0$ 이 되었고, 라우팅 분기가 통째로 삭제된 채 런이 그럴 듯한 퍼플렉시티로 완주했으며 우리는 그것을 기록할 뻔했다. 파싱을 믿지 말고 빈 문자열을 명시적으로 거부하라. (11) Metal이 보고하는 recommendedMaxWorkingSetSize는 설치된 RAM보다 한참 낮다(여기서는 34.4GB 중 26.8GB, iogpu.wired_limit_mb는 기본값). 이 선을 넘는 방식은 균일하지 않다: 수 GB를 크게 넘기면 kIOGPUCommandBufferCallbackErrorOutOfMemory가 나지만, 아슬아슬하게 넘기면 — 가중치는 한도 안, 프리필 연산 버퍼는 한도 밖 — 오류 없이 NaN이 나오고, 같은 가중치의 배치 1 디코드 경로는 수치적으로 정확한 채로 남는다. 순서를 드러낸 것은 접기 배수 스왑이다; 배치 크기만 훑었다면 커널 버그로 오진했을 것이다. 다만 이 스왑은 깨끗한 단일 변수 실험이 아니다 — R을 바꾸면 접기 사상, 유일 블록 수, 로더가 건너뛰는 텐서 집합이 바이트 총량과 함께 바뀐다 — 그래서 §4.2는 이를 진단이 아니라 순서로 보고한다. 크기를 실제로 분리하는 유일한 쌍은 공유 전문가 손잡이를 켜고 끈 R=3이다: 0.4GB 차이로 하나는 유한, 하나는 NaN.